Lab 4: Gaussian Processes

Linus Göransson - lingo978

2026-10-11

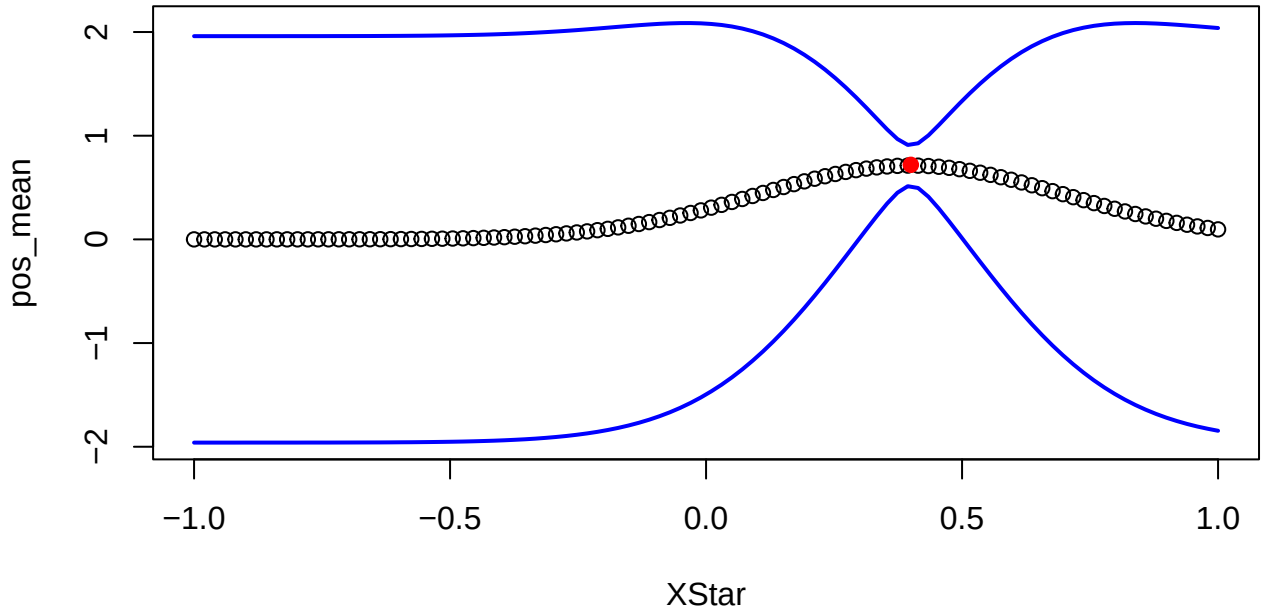
2.1 Implementing GP Regression

(1) posteriorGP

`posteriorGP` implements Algorithm 2.1 from Rasmussen and Williams. The kernel matrix plus noise, $K(X, X) + \sigma_n^2 I$, is Cholesky decomposed (`chol` returns the upper triangular factor, so it is transposed). Then $\alpha = L^T \backslash (L \backslash y)$ gives the posterior mean $\bar{f}_* = K(X, X_*)^T \alpha$, and $v = L \backslash K(X, X_*)$ gives the posterior covariance $K(X_*, X_*) - v^T v$. The function returns the mean and the diagonal of the covariance (the pointwise variances). The kernel is a separate function taken from "GassianProcesses.r" file given. The code is in Appendix A.1.

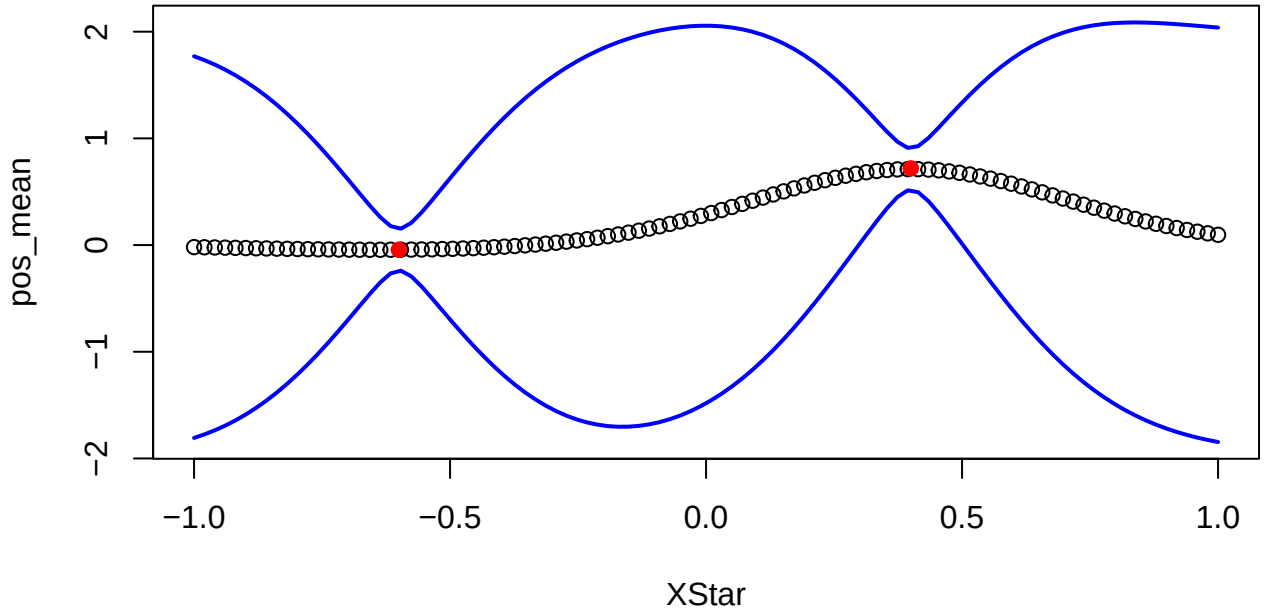
(2) One observation

The prior has $\sigma_f = 1$, $\ell = 0.3$ and $\sigma_n = 0.1$, and is updated with the single observation $(0.4, 0.719)$. In all plots the black points are the posterior mean of f, the blue lines are the 95% pointwise probability bands and the red points are the observations.



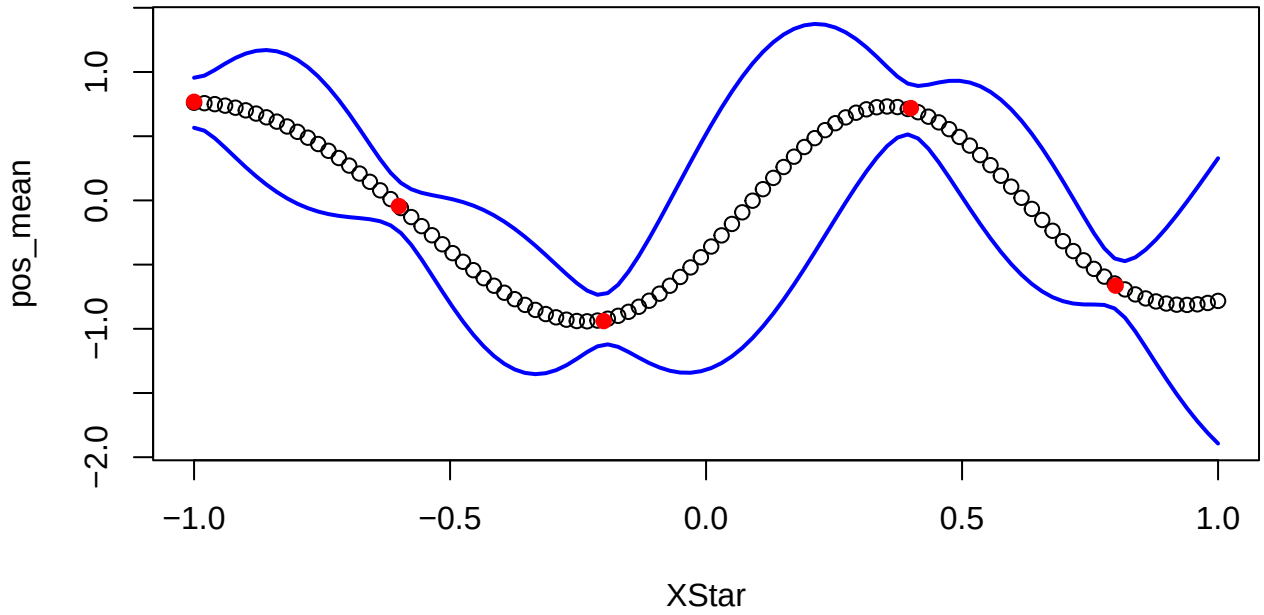
Close to $x = 0.4$ the posterior mean goes up to the observation and the band is narrow. Further away the observation has almost no influence, so the mean goes back to the prior mean 0 and the band goes back to the prior $\pm 1.96\sigma_f$.

(3) Two observations



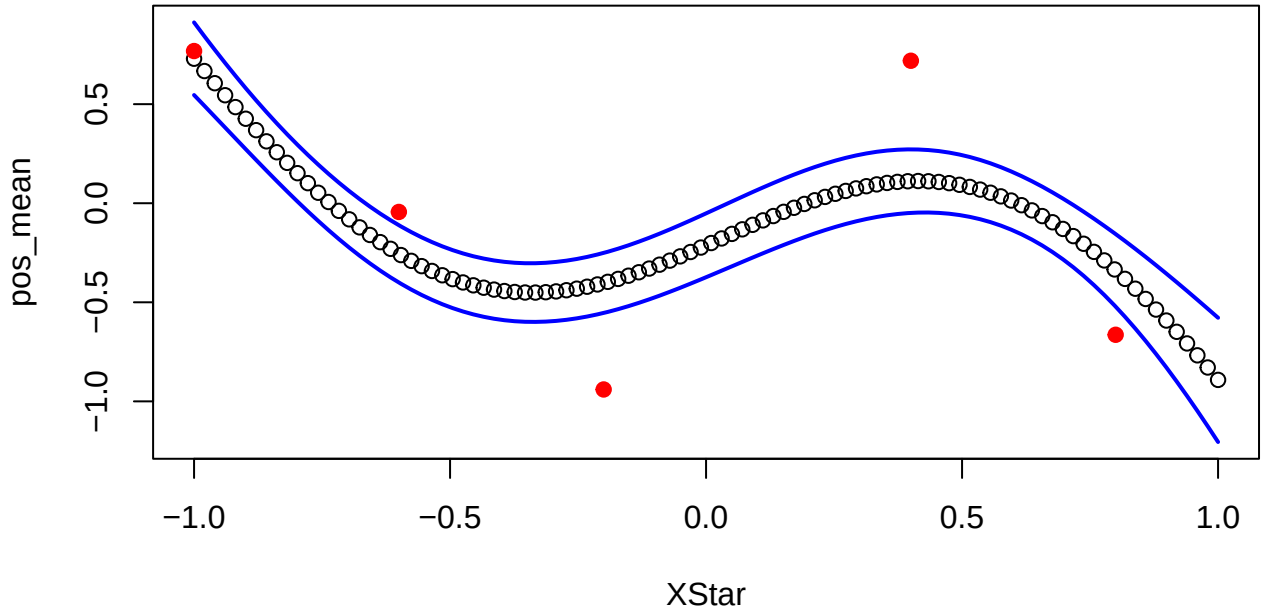
With the second observation $(-0.6, -0.044)$ the band also gets narrow around $x = -0.6$. But still a lot of uncertainty between the two points and at the edges.

(4) Five observations



With all five points the mean goes through (close to) every observation and the bands are narrow near them. Between the points the band widens again.

(5) Five observations, $\ell = 1$



With $\ell = 1$ the function is much smoother, since points far apart are still strongly correlated. The mean no longer goes through the observations, and it can't follow the quick changes between -1.0, -0.6 and -0.2. The bands are narrower everywhere, but this is because the model is more confident than it should be. The posterior mean misses the data by more than the noise level $\sigma_n = 0.1$, so $\ell = 1$ is too large and underfits the data.

2.2 GP Regression with kernlab

The temperature data is subsampled to every fifth day, which gives 438 observations with `time` $= 1, 6, \ldots, 2186$ and `day` $= 1, 6, \ldots, 361$ repeated six times.

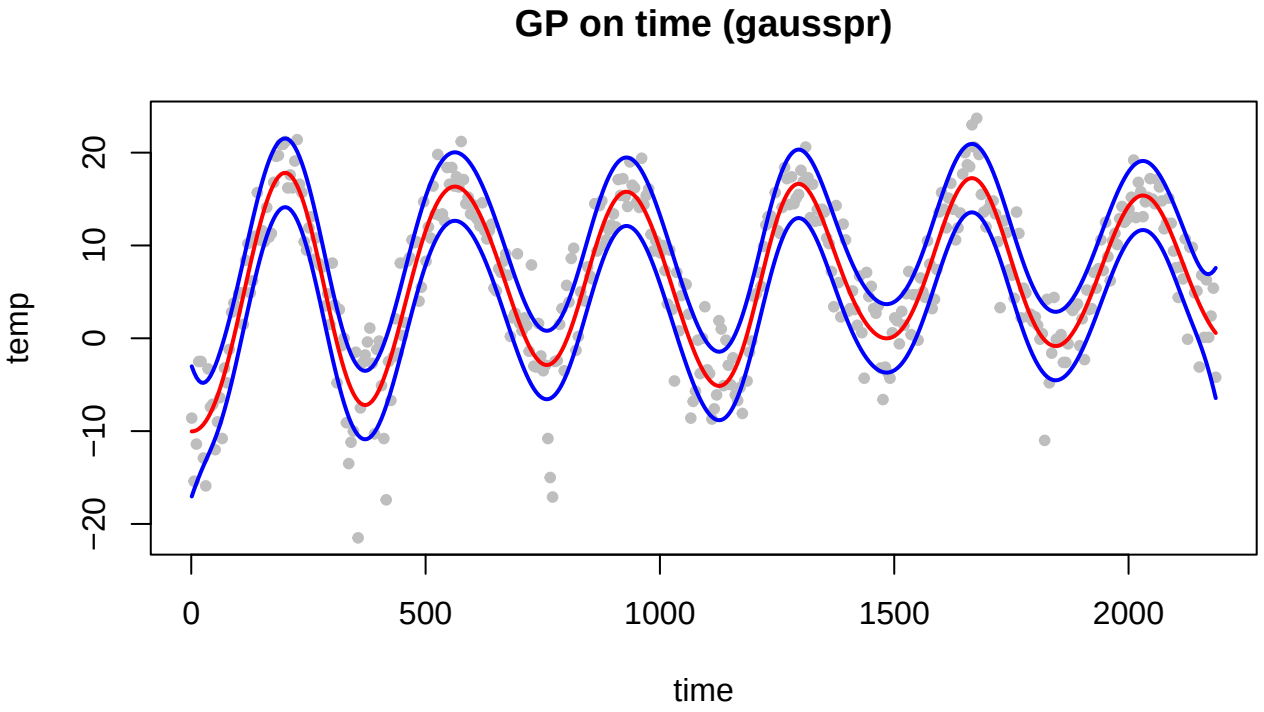
(1) Own squared exponential kernel

```
##            [,1]
## [1,] 0.00386592
```

```
## An object of class "kernelMatrix"
##              [,1]         [,2]        [,3]
## [1,] 3.865920e-03 2.233631e-10 1.92875e-22
## [2,] 3.865920e-03 1.000000e+00 3.86592e-03
## [3,] 2.233631e-10 3.865920e-03 1.00000e+00
```

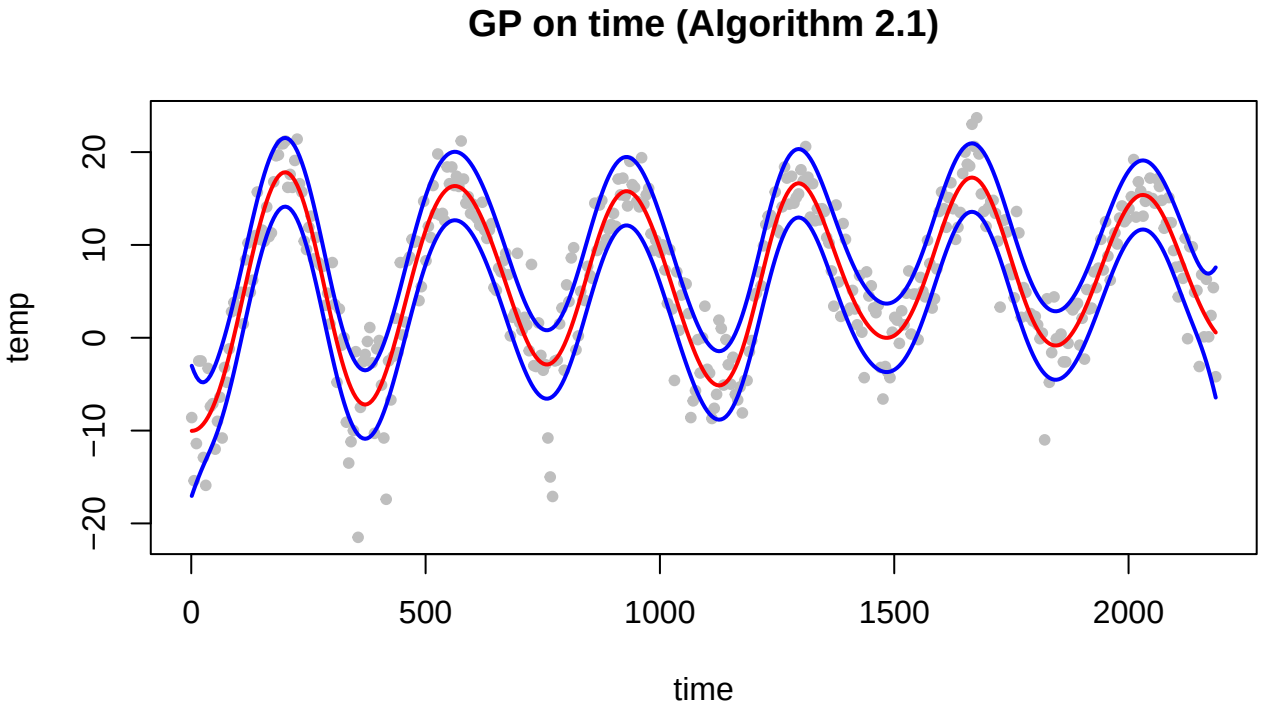
`SEKernel(sigmaF, l)` returns a kernel function of class `kernel`, so it can be used by `kernelMatrix` and `gausspr`. Here it is evaluated with $\sigma_f = 1$, $\ell = 0.3$. $k(1,2) = \exp(-0.5 \cdot (1/0.3)^2) \approx 0.0039$, and `kernelMatrix` gives $K(X, X_*)$ for $X = (1,3,4)^T$ and $X_* = (2,3,4)^T$. The diagonal entries for the shared points 3 and 4 are $\sigma_f^2 = 1$, and the covariance quickly goes to 0 as the distance grows since ℓ is small.

(2) GP on time with gausspr



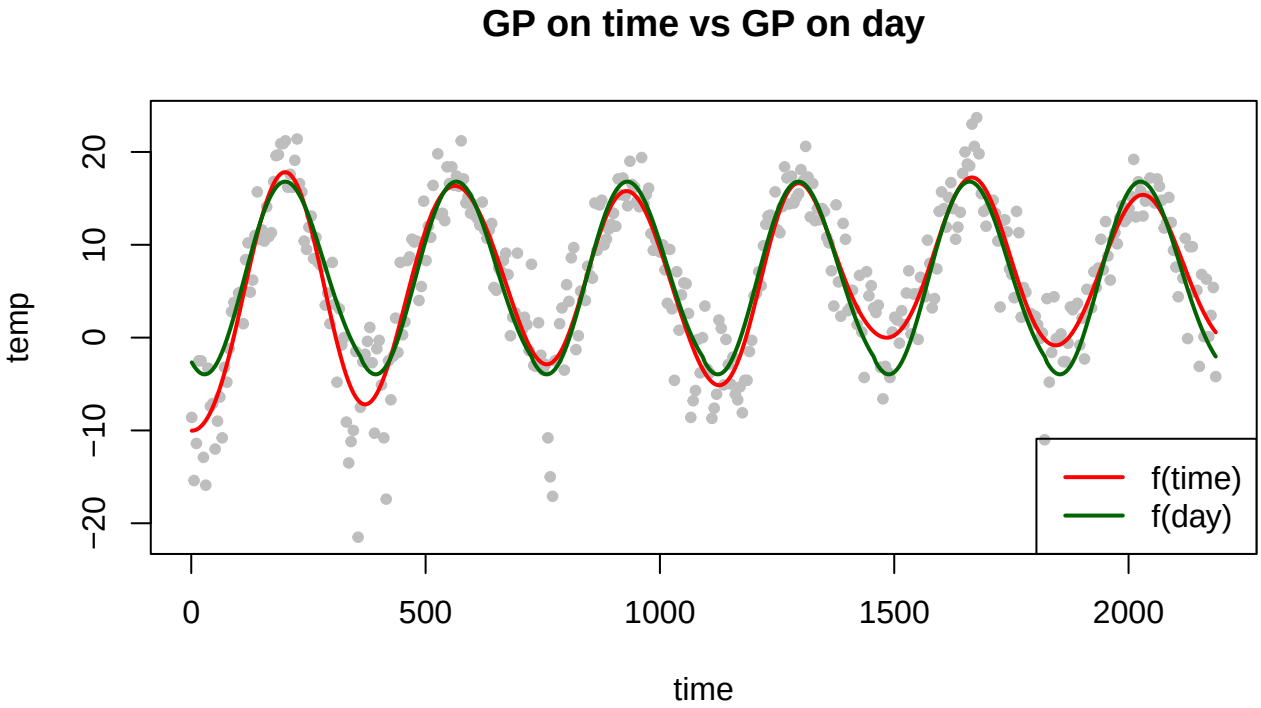
The noise variance is taken from the residuals of a quadratic regression of temp on time, which gives $\sigma_n \approx 8.18$. The GP is fit with $\sigma_f = 20$, $\ell = 100$ and `scaled=FALSE`. The posterior mean (red) follows the seasonal cycle well and the 95% bands for f (blue) are about ± 3.7 degrees around it, wider at the two ends where there is data only on one side. The bands are for f and not for new observations, so a lot of data points fall outside them.

(3) GP on time with Algorithm 2.1



Here the same model is computed with `posteriorGP` from 2.1 (Algorithm 2.1) and the same σ_n, σ_f and ℓ. The result is identical to the `gausspr` fit, the posterior means and standard deviations differ by less than 10^{-12}.

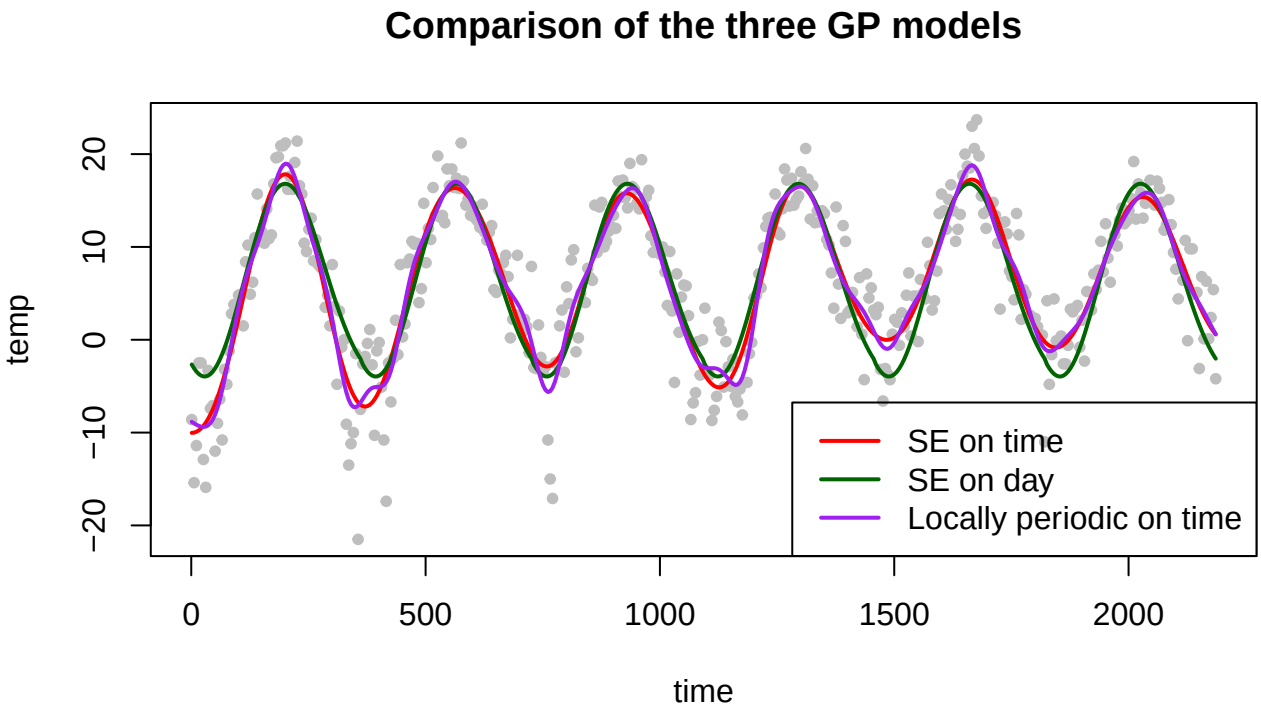
(4) GP on day



Now f is a function of `day`, so all six years share the same function. $\sigma_n \approx 4.85$ is estimated from a quadratic regression on day, which fits much better than on time since a parabola can roughly follow one year. The posterior mean of the day model (green) is plotted on the time axis together with the time model (red).

The day model gives exactly the same curve every year. The pros are that it uses data from all six years to estimate each day, so it is less affected by single extreme years, and it can predict any future year. The con is that it can't model differences between years or any trend. The time model can follow each year separately, but every point only uses data from nearby days, so it can't learn that the pattern repeats and it would not be able to predict far into the future.

(5) Locally periodic kernel



The locally periodic kernel combines both models. The periodic part with $d = 365$ makes days exactly one year apart strongly correlated, and $\ell_1 = 1$ controls how smooth the function is within a year. The SE part with $\ell_2 = 100$ makes the correlation decay over time, so the yearly pattern is allowed to change slowly between years. The same σ_n as in (2) is used.

The fit (purple) is close to the time model, it also captures that some winters are colder than others, e.g. it goes lower than the day model in the first two winters. But since it also shares information from the same period in neighbouring years, it would be a better model for extrapolating forward in time than the SE kernel on time. It gets the pros from both of the previous models.

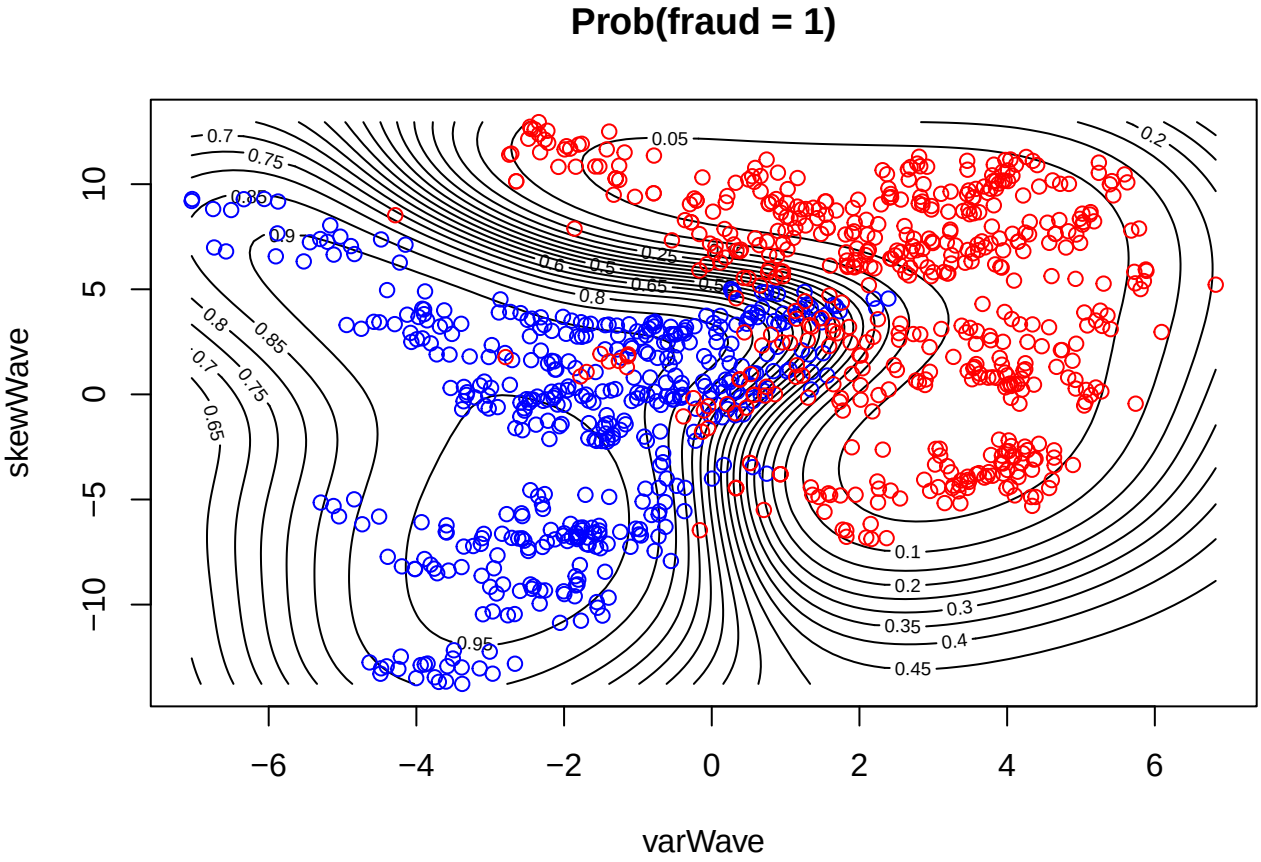
2.3 GP Classification with kernlab

The banknote data has 1372 observations. 1000 of them are used for training and the remaining 372 are used for testing. All models use the default kernel in `gausspr` (Gaussian RBF) with the width σ estimated automatically by `sigest`.

(1) Two covariates

```
## Using automatic sigma estimation (sigest) for RBF or laplace kernel
```

```
## Gaussian Processes object of class "gausspr"
## Problem type: classification
##
## Gaussian Radial Basis kernel function.
##  Hyperparameter : sigma =  1.42415004757028
##
## Number of training instances learned : 1000
## Train error : 0.059
```




```
##
## predTrain   0   1
##         0 503  18
##         1  41 438
```

```
## [1] 0.941
```

The contours show the predicted probability of fraud = 1 over a 100 x 100 grid of `varWave` and `skewWave`, with fraud = 1 training points in blue and fraud = 0 in red. Frauds are mostly found at low `varWave`, and the probability goes from above 0.9 on the left to below 0.1 on the right. There is a clear overlap between the classes in the middle of the plot, where the probabilities are close to 0.5. The training accuracy is 0.941.

(2) Test accuracy, two covariates

```
##
## predTest   0   1
##        0 199   9
```

```
##        1  19 145
```

```
## [1] 0.9247312
```

The test accuracy is 0.925, a bit lower than the training accuracy. Since the drop is small the model does not overfit much, and most of the errors come from the overlap between the classes that can't be separated using only these two covariates.

(3) Four covariates

```
## Using automatic sigma estimation (sigest) for RBF or laplace kernel
```

```
## Gaussian Processes object of class "gausspr"
## Problem type: classification
##
## Gaussian Radial Basis kernel function.
##  Hyperparameter : sigma =  0.438275644976386
##
## Number of training instances learned : 1000
## Train error : 0.003
```

```
##
## predTestAll   0   1
##           0 216   0
##           1   2 154
```

```
## [1] 0.9946237
```

With all four covariates the test accuracy goes up to 0.995, only 2 out of 372 test observations are misclassified. `kurtWave` and `entropyWave` contain information that separates the classes where `varWave` and `skewWave` overlap, so the model with all four covariates is clearly better.

Appendix A: R Code

The code is listed below, grouped by question.

A.1 GP Regression: kernel, posteriorGP and plotting

```
# Squared exponential kernel: k(x, x') = sigmaF^2 * exp(-(x - x')^2 / (2 l^2))
# Returns the n1 x n2 covariance matrix where entry (i, j) = k(x1[i], x2[j])
SquaredExpKernel <- function(x1,x2,sigmaF=1,l=3) {
  n1 <- length(x1) # number of points in x1 (rows of K)
  n2 <- length(x2) # number of points in x2 (columns of K)
  K <- matrix(NA,n1,n2) # empty n1 x n2 matrix to fill in
  for (i in 1:n2){
    # Column i: similarity between every x1 and the single point x2[i]
    K[,i] <- sigmaF^2*exp(-0.5*( (x1-x2[i])/l)^2 )
  }
  return(K)
}

posteriorGP <- function(X, y, XStar, sigmaNoise, k, ...) {
  n <- length(X) # number of training points
  K <- k(X, X, ...)  # K(X, X), n x n: covariance between training points
  kStar <- k(X, XStar, ...) # K(X, X*), n x m: covariance between training and test points

  # Algorithm 2.1
  noise_diag <- sigmaNoise^2 * diag(n) # sigma^2*I, noise variance on the diagonal
  # Cholesky decomposition. R returns upper triangular matrix so we transpose
  L <- t(chol(K + noise_diag))

  alpha <- solve(t(L), solve(L, y))
  # Posterior mean of f at XStar
  fStar <- t(kStar) %*% alpha

  v <- solve(L, kStar)
  # Posterior covariance of f at XStar
  V_fStar <- k(XStar, XStar, ...) - t(v) %*% v

  # Log marginal likelihood, not needed?
  log_lik <- -1/2*t(y)%*%alpha - sum(log(diag(L))) - n/2*log(2*pi)

  return(list(mean = as.vector(fStar), variance = diag(V_fStar)))
}

plot_gp <- function(XStar, pos_mean, pos_variance, X, y) {
  # 95% confidence intervals
  lower_bound <- pos_mean - 1.96*sqrt(pos_variance)
  upper_bound <- pos_mean + 1.96*sqrt(pos_variance)

  # plot posterior mean
  plot(XStar, pos_mean, type="p", ylim = range(c(lower_bound, upper_bound)))

  # plot confidence intervals
  lines(XStar, lower_bound, col = "blue", lwd = 2)
  lines(XStar, upper_bound, col = "blue", lwd = 2)

  # plot training data points
  points(X, y, pch = 19, col = "red")
}
```

A.2 GP Regression: questions 2.1 (2)-(5)

```
# (x, y) = (0.4, 0.719)
X <- c(0.4)
y <- c(0.719)
# interval x in [-1, 1]
XStar <- seq(-1, 1, length.out = 100)

res <- posteriorGP(X, y, XStar, sigmaNoise=0.1, SquaredExpKernel, sigmaF=1, l=0.3)
plot_gp(XStar, res$mean, res$variance, X, y)
```

```
# updating prior data
X <- c(0.4, -0.6)
y <- c(0.719, -0.044)

# same hyperparams
res <- posteriorGP(X, y, XStar, sigmaNoise=0.1, SquaredExpKernel, sigmaF=1, l=0.3)
plot_gp(XStar, res$mean, res$variance, X, y)
```

```
# updating prior data
X <- c(-1, -0.6, -0.2, 0.4, 0.8)
y <- c(0.768, -0.044, -0.940, 0.719, -0.664)

# same hyperparams
res <- posteriorGP(X, y, XStar, sigmaNoise=0.1, SquaredExpKernel, sigmaF=1, l=0.3)
plot_gp(XStar, res$mean, res$variance, X, y)
```

```
# l = 1
res <- posteriorGP(X, y, XStar, sigmaNoise=0.1, SquaredExpKernel, sigmaF=1, l=1)
plot_gp(XStar, res$mean, res$variance, X, y)
```

A.3 GP Regression with kernlab: setup and (1)

```
library(kernlab)

data <- read.csv("https://github.com/STIMALiU/AdvMLCourse/raw/master/GaussianProcess/Code/TempTullinge.csv", h

time <- seq(1, 2190, 5)
day <- rep(seq(1, 365, 5), 6)
temp <- data$temp[time] # subsampled temperatures (every fifth day)
```

```
SEKernel <- function(sigmaF, l) {
  kval = function (X, XStar) {
    K = matrix(NA, length(X), length(XStar))
    for (i in 1:length(XStar)) {
      K[, i] = sigmaF ^ 2 * exp(-0.5 * ((X - XStar[i]) / l) ^ 2)
    }
    return(K)
  }
  class(kval) = 'kernel'
  return (kval)
}

se_kernel <- SEKernel(1, 0.3)
se_kernel(1, 2)

X <- c(1, 3, 4)
XStar <- c(2, 3, 4)
kernelMatrix(kernel = se_kernel, x = X, y = XStar)
```

A.4 GP Regression with kernlab: (2)-(5)

```
# Noise variance from the residuals of a quadratic regression on time
quadFit <- lm(temp ~ time + I(time^2))
sigmaNoise <- sd(quadFit$residuals)

# GP fit with our SE kernel, sigmaF = 20, l = 100
GPfit <- gausspr(time, temp, kernel = SEKernel, kpar = list(sigmaF = 20, l = 100),
                 var = sigmaNoise^2, variance.model = TRUE, scaled = FALSE)
meanPred <- predict(GPfit, time)
sdPred <- predict(GPfit, time, type = "sdeviation")

plot(time, temp, pch = 20, col = "grey", main = "GP on time (gausspr)")
lines(time, meanPred, col = "red", lwd = 2)
# 95% bands for f
lines(time, meanPred - 1.96*sdPred, col = "blue", lwd = 2)
lines(time, meanPred + 1.96*sdPred, col = "blue", lwd = 2)
```

```
# Same model, but with own implementation of Algorithm 2.1 from 2.1
res <- posteriorGP(time, temp, time, sigmaNoise = sigmaNoise, SquaredExpKernel, sigmaF = 20, l = 100)

plot(time, temp, pch = 20, col = "grey", main = "GP on time (Algorithm 2.1)")
lines(time, res$mean, col = "red", lwd = 2)
# 95% bands for f
lines(time, res$mean - 1.96*sqrt(res$variance), col = "blue", lwd = 2)
lines(time, res$mean + 1.96*sqrt(res$variance), col = "blue", lwd = 2)
```

```
# Noise variance from a quadratic regression on day
quadFitDay <- lm(temp ~ day + I(day^2))
sigmaNoiseDay <- sd(quadFitDay$residuals)

GPfitDay <- gausspr(day, temp, kernel = SEKernel, kpar = list(sigmaF = 20, l = 100),
                    var = sigmaNoiseDay^2, scaled = FALSE)
meanPredDay <- predict(GPfitDay, day)

# Both posterior means with time on the horizontal axis
plot(time, temp, pch = 20, col = "grey", main = "GP on time vs GP on day")
lines(time, meanPred, col = "red", lwd = 2)
lines(time, meanPredDay, col = "darkgreen", lwd = 2)
legend("bottomright", legend = c("f(time)", "f(day)"), col = c("red", "darkgreen"), lwd = 2)
```

```
# Locally periodic kernel: periodic part (l1, d) times SE part (l2)
PeriodicKernel <- function(sigmaF, l1, l2, d) {
  kval = function (X, XStar) {
    K = matrix(NA, length(X), length(XStar))
    for (i in 1:length(XStar)) {
      K[, i] = sigmaF ^ 2 * exp(-2 * sin(pi * abs(X - XStar[i]) / d) ^ 2 / l1 ^ 2) *
                            exp(-0.5 * ((X - XStar[i]) / l2) ^ 2)
    }
    return(K)
  }
  class(kval) = 'kernel'
  return (kval)
}

GPfitPer <- gausspr(time, temp, kernel = PeriodicKernel,
                    kpar = list(sigmaF = 20, l1 = 1, l2 = 100, d = 365),
                    var = sigmaNoise^2, scaled = FALSE)
meanPredPer <- predict(GPfitPer, time)
```

```r
# Compare all three models
plot(time, temp, pch = 20, col = "grey", main = "Comparison of the three GP models")
lines(time, meanPred, col = "red", lwd = 2)
lines(time, meanPredDay, col = "darkgreen", lwd = 2)
lines(time, meanPredPer, col = "purple", lwd = 2)
legend("bottomright", legend = c("SE on time", "SE on day", "Locally periodic on time"),
       col = c("red", "darkgreen", "purple"), lwd = 2)
```

A.5 GP Classification with kernlab

```r
data <- read.csv("https://github.com/STIMALiU/AdvMLCourse/raw/master/GaussianProcess/Code/banknoteFraud.csv",
names(data) <- c("varWave","skewWave","kurtWave","entropyWave","fraud")
data[,5] <- as.factor(data[,5])

set.seed(111); SelectTraining <- sample(1:dim(data)[1], size = 1000, replace = FALSE)
train <- data[SelectTraining, ]
test <- data[-SelectTraining, ]
```

```r
# GP classification with default kernel (rbfdot) and hyperparameters
GPfitFraud <- gausspr(fraud ~ varWave + skewWave, data = train)
GPfitFraud

# Grid over varWave and skewWave
x1 <- seq(min(train$varWave), max(train$varWave), length = 100)
x2 <- seq(min(train$skewWave), max(train$skewWave), length = 100)
gridPoints <- expand.grid(varWave = x1, skewWave = x2) # x1 varies fastest

# Probability of fraud = 1 over the grid
probPreds <- predict(GPfitFraud, gridPoints, type = "probabilities")
contour(x1, x2, matrix(probPreds[, "1"], 100, 100), 20,
        xlab = "varWave", ylab = "skewWave", main = "Prob(fraud = 1)")
points(train$varWave[train$fraud == 1], train$skewWave[train$fraud == 1], col = "blue")
points(train$varWave[train$fraud == 0], train$skewWave[train$fraud == 0], col = "red")

# Confusion matrix and accuracy on the training data
predTrain <- predict(GPfitFraud, train[, 1:2])
confTrain <- table(predTrain, train$fraud)
confTrain
sum(diag(confTrain)) / sum(confTrain)
```

```r
# Same model on the test set
predTest <- predict(GPfitFraud, test[, 1:2])
confTest <- table(predTest, test$fraud)
confTest
sum(diag(confTest)) / sum(confTest)
```

```r
# Model with all four covariates
GPfitFraudAll <- gausspr(fraud ~ ., data = train)
GPfitFraudAll

predTestAll <- predict(GPfitFraudAll, test[, 1:4])
confTestAll <- table(predTestAll, test$fraud)
confTestAll
sum(diag(confTestAll)) / sum(confTestAll)
```